Formal Verification meets AI

Using Lean as a Practical Bridge

Aziz Najjar

Formal Semantics, Spring 2026
Instructor: Dániel Horpácsi
ELTE Faculty of Informatics

Abstract

Core idea

Formal methods from this course apply directly to verifying AI systems. Lean 4 provides a practical bridge between theory and practice.

Roadmap:

- An AI failure that testing could not prevent
- Testing vs. formal proof — the fundamental gap
- Three verified Lean 4 properties of neural network components
- How each example connects to course concepts
- Where the field is heading: LeanDojo, AlphaProof, DeepSeek-Prover-V2

A Note on Tools

Retrieval (Claude + RAG)

Lecture PDFs 1, 3, 4 uploaded as context. Used to ground references to:

- compositionality
- static semantics
- derivation sequences

Lean 4 proofs

- Inspired by *Theorem Proving in Lean 4* (Avigad et al.)
- **All proofs written and verified manually** in VS Code

Next: what retrieval over lecture PDFs looks like in practice.

Querying Lecture Material

Claude generated these lecture summaries by querying Lectures 1, 3, and 4 as project knowledge — each diagram is grounded in the actual PDF content, not general knowledge.

Why Testing Is Not Enough

Dijkstra (1969)

"Testing shows the presence of bugs, not their absence."

Amazon Rekognition (2018)

28 US Congress members misidentified as criminals after extensive testing — the model was evaluated on the wrong distribution of inputs.

Testing

- Finite set of inputs
- One missed case can fail

Formal proof

- Guaranteed for *all* $x \in \mathbb{R}$
- No exceptions possible

aclu.org/news/privacy-technology/amazons-face-recognition-falsely-matched-28

From Semantics to Lean

Course concept	Lean counterpart
Inference rules (Lec. 3, 4)	Tactics apply rules step-by-step — a derivation tree
Compositionality (Lec. 3)	Proofs compose from sub-proofs, like $[\![e_1{+}e_2]\!]s$
Static semantics (Lec. 1)	Dependent types enforce constraints at compile time
States: $\mathrm{Var} \to \mathbb{Z}$ (Lec. 3)	Lean's local context plays the role of the state function

Key idea: course definitions become **checkable proof obligations**. Lec. 3 writes $\mathcal{A}[\![a_1{+}a_2]\!]s = \mathcal{A}[\![a_1]\!]s + \mathcal{A}[\![a_2]\!]s$ on paper; Lean enforces it.

Example 1 / 3 ReLU is Non-Negative

relu.lean

```
noncomputable def relu (x : R) : R :=
  max 0 x

theorem relu_nonneg (x : R) :
    relu x >= 0 := by
  unfold relu         -- expand definition
  exact le_max_left 0 x -- 0 <= max 0 x
```

`unfold relu` — substitution, like β-reduction

`exact le_max_left` — Mathlib: $0 \leq \max(0, x)$

Course link: universal quantification $\leftrightarrow$ $[\![a]\!]s$ *defined for all states s (Lec. 3)*

Why it matters

ReLU is the standard NN activation. This proof holds for *all* $x \in \mathbb{R}$ — no test suite can do that.

Example 2 / 3 Vector Dimension Safety

vecadd.lean

```
def vecAdd {n : N} (a b : Fin n -> R)
    : Fin n -> R :=
  fun i => a i + b i

def v3 : Fin 3 -> R := ![1, 2, 3]
def v2 : Fin 2 -> R := ![4, 5]

#check vecAdd v3 v3  -- OK:  Fin 3 -> R
-- #check vecAdd v3 v2
-- ^ TYPE ERROR at compile time
```

Why it matters

Shape mismatch is the #1 crash in PyTorch/TensorFlow. Lean catches it *before* the program runs.

Dependent types: `Fin n -> R` carries its length in the type itself — no runtime check needed.

Course link: static semantics (Lec. 1) — constraints beyond context-free grammars

Example 3 / 3 MSE Loss is Non-Negative

mse.lean

```
theorem mse_nonneg {n : N}
    (ys ys' : Fin n -> R) :
    0 <= sum i, (ys i - ys' i) ^ 2 := by
  apply Finset.sum_nonneg
  -- sum of >=0 terms is >=0
  intro i _
  exact sq_nonneg _
  -- (y_i - y'_i)^2 >= 0
```

Why it matters

MSE is the most common regression loss. A verified lower bound of 0 eliminates a class of numerical instability bugs.

Compositional proof: *sq_nonneg* $\to$ *sum_nonneg* $\to$ *mse_nonneg* — mirrors $[\![e_1+e_2]\!]s = [\![e_1]\!]s + [\![e_2]\!]s$

Course link: compositionality (Lec. 3)

Try It Yourself

GitHub repository

`github.com/Njoura7/formal-verification-ai`

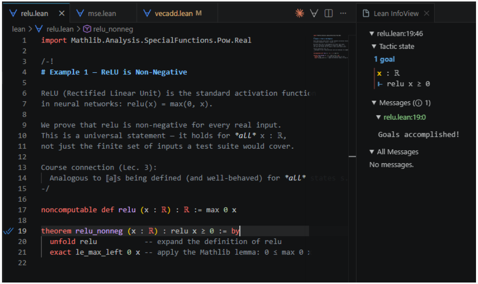
What is there

- `lean/relu.lean`
- `lean/vecadd.lean`
- `lean/mse.lean`
- `README.md` with setup steps

To run in VS Code

1. Install `leanprover.lean4` extension
2. `lake exe cache get`
3. Open any `.lean` file
4. See Goals accomplished!

All three proofs compile against Mathlib — reproducible from a fresh install.



What you see

Left — proof file with tactics

Right — Lean Infoview:

- Tactic state shows the open goal
- $\mathit{relu}\ \ x \geq 0$ waiting to be closed
- **Goals accomplished!** once all tactics apply

When AI Starts Proving Things

LeanDojo
NeurIPS 2023

Treats Lean's proof state as an environment. A model learns to select the next tactic — a derivation sequence, step by step.

AlphaProof
DeepMind, 2024

Formalised IMO problems in Lean and trained on proof search. Solved 4 of 6 problems at silver medal level.

DeepSeek-Prover-V2
2025

Splits goals into sub-goals before proving — the same compositional breakdown as denotational semantics.

These systems do not reason in prose — they work inside a formal language where every step is checkable. The semantics is the guarantee.

Conclusion

- Testing is incomplete — a proof is not
- Lean tactics *are* derivation sequences from the course
- ReLU, MSE, and shape safety are formally provable today
- AI systems are beginning to find those proofs themselves

Takeaway

AI systems can move from *working* to provably correct.

References i

- Yang, K. et al. *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models.* NeurIPS 2023. `arXiv:2306.15626`
- DeepMind. *AI achieves silver-medal standard solving IMO problems.* Blog post, July 2024. `deepmind.google`
- Ying, H. et al. *DeepSeek-Prover-V2.* `arXiv:2504.21801`, 2025.
- ACLU. *Amazon's Face Recognition Falsely Matched 28 Members of Congress.* 2018. `aclu.org`
- Horpácsi, D. *Formal Semantics* — Lecture Notes, Lec. 1, 3, 4. ELTE, Spring 2026.
- Avigad, J. et al. *Theorem Proving in Lean 4.* `leanprover.github.io`

Questions?